МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа №1**
**по курсу “Компьютерная графика”**

***Основы 3D графики***

Выполнил: И. М. Шагалиев
Группа: М9О-302БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

## Условие

Познакомиться с основами 3D-графики: построением простых 3D-объектов, проекцией на 2D-плоскость, работой с матрицами перспективы, ортографической проекции и аффинными преобразованиями.

**Вариант 7 — Тор.**

Требуется реализовать отображение тора — параметрической 3D-фигуры, образованной вращением окружности радиуса r вокруг оси, лежащей в её плоскости на расстоянии R от центра. Программа должна работать в реальном времени, позволять динамически менять проекцию и трансформации объекта, а также взаимодействовать с пользователем через интерфейс.

Выполнены дополнительные задания 1–5:

1. Переключение ортографической и перспективной проекций.
2. Изменение позиции, поворота и масштаба фигуры.
3. Анимация по сложной траектории с управлением паузой, скоростью и радиусом.
4. Изменение цвета фигуры через элементы интерфейса.
5. Процедурные цвета вершин, зависящие от их локальных координат.

## Метод решения

Построение геометрии тора. Тор задаётся параметрически через два угла: u — вокруг главной оси, v — вокруг оси трубы. Координаты вершины:

$$x = (R + r\cdot\cos v)\cdot\cos u$$
$$y = r\cdot\sin v$$
$$z = (R + r\cdot\cos v)\cdot\sin u$$

Параметры: R = 1.0, r = 0.35, N = 64 сегментов по главной окружности, M = 32 сегмента по трубе. Всего получается 2145 вершин и 12288 индексов. Каждая вершина имеет позицию и цвет, вычисляемый процедурно по её локальным координатам:

$$color = (0.5 + 0.5\cdot\sin(2x),\ 0.5 + 0.5\cdot\sin(2y),\ 0.5 + 0.5\cdot\sin(2z))$$

Буферы вершин и индексов. Геометрия тора загружается в GPU через Vulkan Memory Allocator (VMA). Vertex buffer хранит позицию и цвет, index buffer — порядок обхода вершин для треугольников.

Uniform buffer и MVP-матрицы. Каждый кадр вычисляются три матрицы через библиотеку glm:

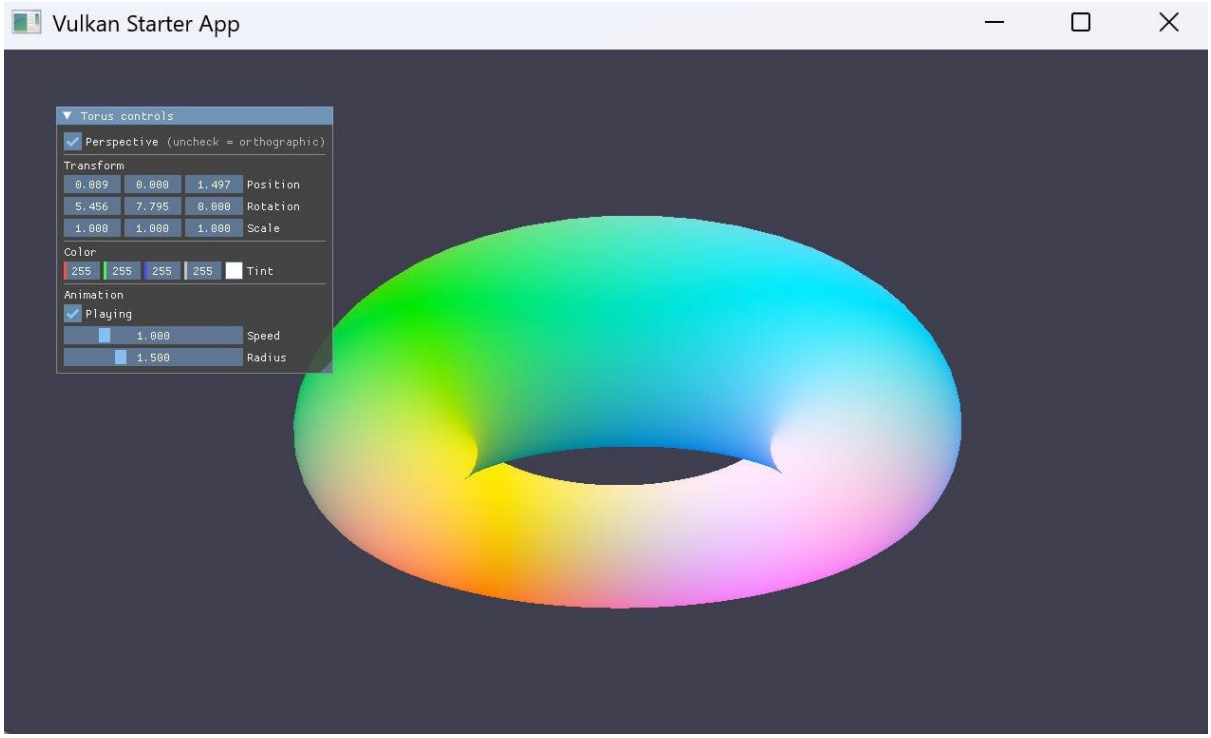
- Model — позиция, поворот и масштаб тора, задаваемые пользователем.
- View — камера, расположенная в точке (0, 1, 4), смотрит в центр сцены.
- Projection — перспективная (glm::perspective, угол 60°) или ортографическая (glm::ortho), в зависимости от режима.

Итоговая матрица *proj · view · model* умножается на позицию вершины в вершинном шейдере и передаётся в стандартизованные координаты экрана. Матрицы записываются в uniform buffer, доступный шейдеру через descriptor set.

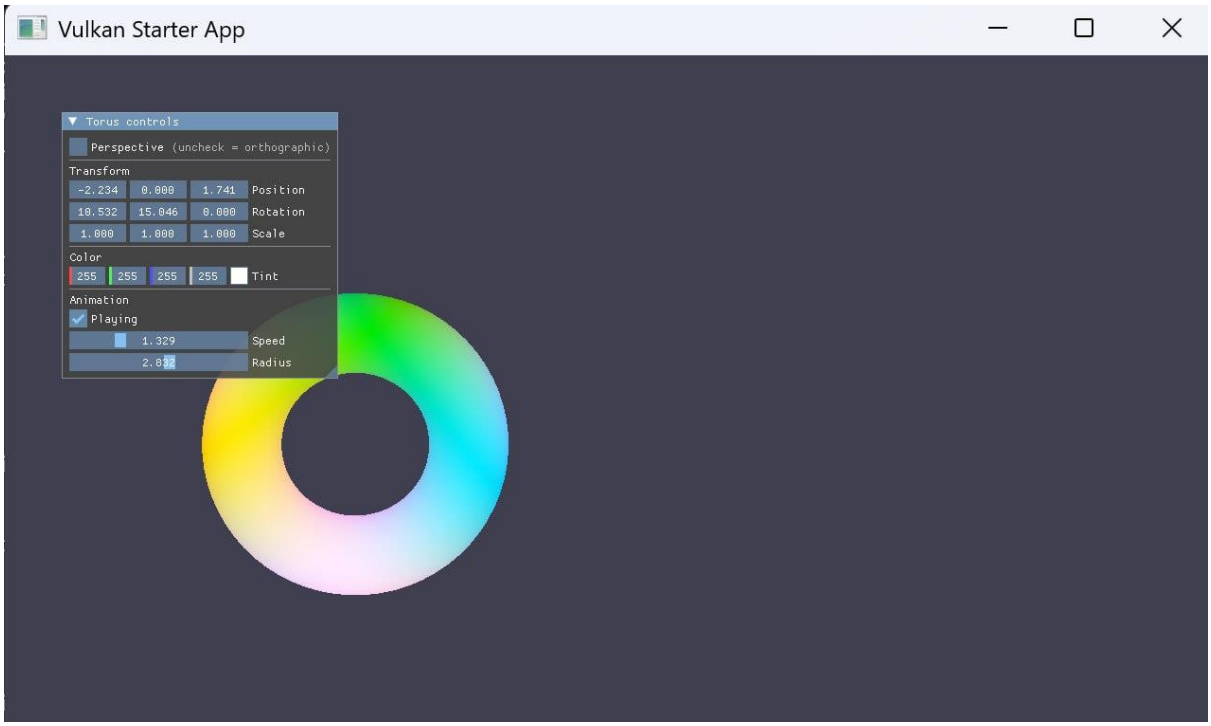
Шейдеры. Вершинный шейдер (torus.vert) применяет MVP-матрицы и передаёт цвет вершины во фрагментный. Фрагментный шейдер (torus.frag) умножает процедурный цвет вершины на параметр tint — цвет, выбираемый пользователем в интерфейсе, — и выводит результат. tint передаётся через push constant, чтобы не пересоздавать descriptor set при каждом изменении цвета.

Используемые объекты Vulkan. Render pass с двумя вложениями (цвет и depth buffer формата D24_UNORM_S8_UINT); graphics pipeline с включёнными depth-тестом и back-face culling; command buffer, в который записываются команды vkCmdBeginRenderPass, vkCmdBindPipeline, vkCmdBindDescriptorSets, vkCmdBindVertexBuffers, vkCmdBindIndexBuffer, vkCmdDrawIndexed; семафоры и fence для синхронизации кадров. Синхронизация построена на одном кадре в полёте. Интерфейс реализован на ImGui и рендерится отдельным render pass-ом поверх сцены.

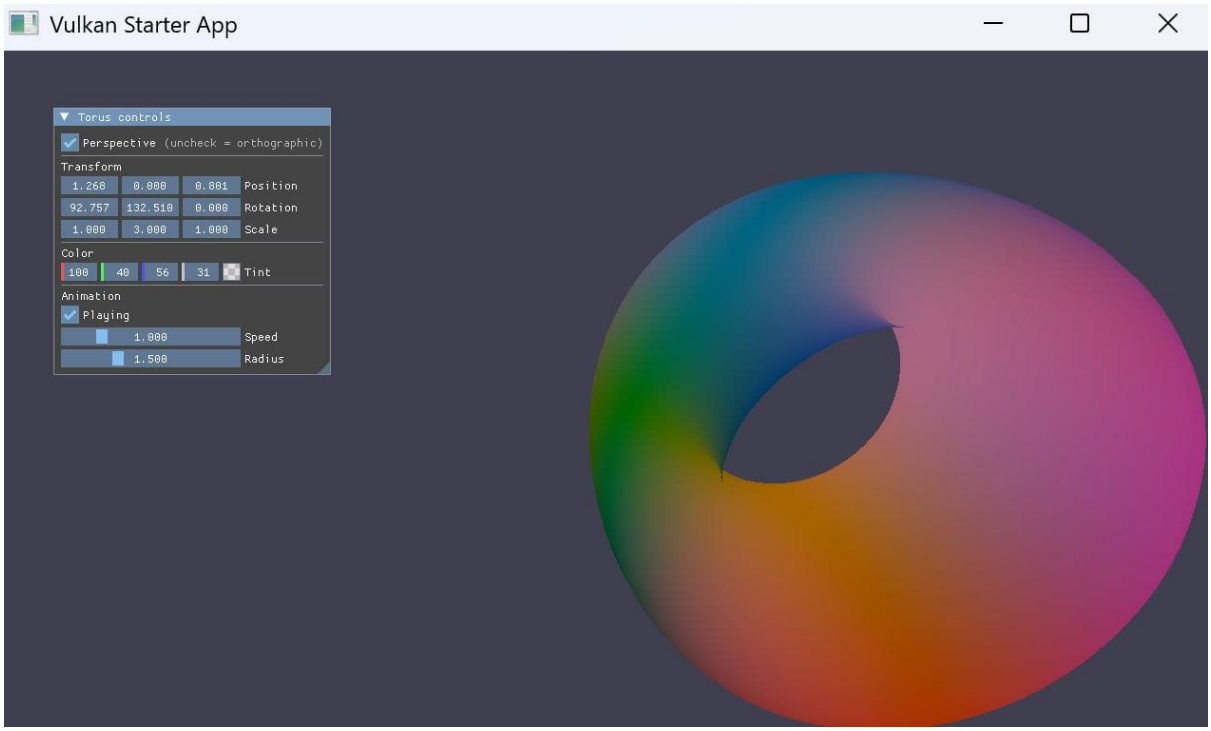
## Результаты




1. Тор в перспективной проекции. (Тор отображается объёмно, видна глубина: задняя часть трубы перекрывается передней. Цвета радужные — это процедурные цвета вершин, зависящие от их локальных координат.)




2. Тор в ортографической проекции. (После снятия галочки «Perspective» в интерфейсе тор отображается без перспективного сокращения — параллельные линии остаются параллельными.)



3. Изменение цвета и трансформация. (С помощью ColorEdit4 изменён цвет тора; с помощью Position / Rotation / Scale изменены положение, угол поворота и размеры.)

## Выводы

Я научился создавать и отображать трёхмерные объекты средствами Vulkan, работать с матрицами перспективной и ортографической проекции, выполнять аффинные преобразования (перенос, поворот, масштабирование) через матрицу модели, а также передавать данные в шейдеры через uniform buffer и push constants. Освоил базовую работу с графическим конвейером Vulkan: создание render pass, graphics pipeline, дескрипторов, командных буферов и синхронизацию кадров. Дополнительно познакомился с процедурной генерацией цвета вершин и интеграцией интерфейса ImGui для управления сценой в реальном времени.